

Sistema de Calibracion Angular

Sensor ILD1302 + Plato Rotatorio

Documentacion tecnica de uso y configuracion

Mayo 2026 | v1.0

Mauricio Alejandro Diaz Barrera

Perfil Github: <https://github.com/ApocalypZmbe>

Indice

Indice	2
1. Vision general del sistema	3
2. Estructura de archivos	3
Componentes por carpeta.....	3
3. Flujo de uso paso a paso	4
3.1. Modo Automatico (Arduino autonomo)	4
3.2. Modo Semi-Automatico (boton fisico)	4
3.3. Modo Manual (Monitor Serie)	4
Comandos en modo manual	4
4. Configuracion del Arduino (.ino).....	6
5. Configuracion del script Python (.py).....	6
6. Casos de uso: alternar para distintas maquinas.....	7
6.1. Tabla de combinaciones tipicas.....	7
6.2. Como elegir el paso angular (PASO_ANGULO_DEG)	8
6.3. Como elegir el tiempo por posicion.....	8
6.4. Como elegir velocidad (VEL_US)	8
7. Logica interna del sistema.....	9
7.1. Compensacion acumulativa de pasos	9
7.2. Deteccion de periodos quietos en el Python	9
7.3. Asignacion de periodos a posiciones.....	9
7.4. Calculo de resultados	10
Tabla de calibracion.....	10
Tabla de reposicionamiento (solo si se miden ambos sentidos).....	10
8. Solucion de problemas.....	10
El motor no se mueve	10
El motor vibra pero no avanza	10
El plato no llega exactamente a 360° despues de una vuelta	10
Python no detecta todas las pausas.....	10
Python detecta MAS pausas de las esperadas	10
El Excel sale con N/D en muchas celdas	11
9. Resumen rapido.....	11

1. Vision general del sistema

Este sistema permite calibrar la posicion angular de un plato rotatorio midiendo la distancia con un sensor laser triangulacion Micro-Epsilon ILD1302. La idea es: el plato gira en saltos angulares iguales, el sensor mide la distancia a una superficie de referencia en cada parada, y un script de Python analiza los datos para reportar errores y desviaciones por posicion.

El sistema admite tres modos de operacion segun el grado de automatizacion deseado:

Aspecto	Semi-Automatico	Automatico	Manual
Quien mueve el plato	Operador con boton	Arduino solo	Operador via PC
Paso angular	Lo que da el boton (90°)	Configurado en .ino (30°)	Configurado en .ino
Tiempo por posicion	≥3s (humano)	Exactamente 3s	Lo que tarde el operador
Repetibilidad	Media (variable)	Alta (deterministica)	Media-Baja
Riesgo de saltar pos	Alto	Nulo	Medio
Util cuando	No quieres modificar el .ino	Ensayo formal de calibracion	Pruebas puntuales

Importante: el mismo script de Python sirve para los tres modos. La diferencia esta en el .ino (Arduino) y en como se opera el ensayo. El analizador detecta los periodos quietos sin importar quien los genero.

2. Estructura de archivos

El sistema esta organizado en una sola carpeta con los tres archivos principales:

```

Medicion ILD1302/
├── analizador.py          Script de analisis del CSV
├── lanzar.bat            Lanzador para Windows (doble clic)
├── Documentacion.docx    Este archivo
└── ILD1302/
    ├── ILD1302/
    │   └── ILD1302.ino    Sketch del Arduino
    └──

```

Componentes por carpeta

- .ino - Sketch que se carga al Arduino. Controla el motor paso a paso.
- .py - Script que lee el CSV del Setup Tool y genera el Excel + grafica.
- .bat - Lanzador de un clic para Windows. Verifica Python, instala librerias si faltan, y abre el dialogo para elegir el CSV.

3. Flujo de uso paso a paso

3.1. Modo Automatico (Arduino autonomo)

Es el modo recomendado para ensayos formales. El Arduino mueve solo y el operador no interviene durante la medicion.

1. Conecta el Arduino por USB. Asegurate de que el cable USB del PC NO este abriendo el monitor serie del IDE.
2. Abre el Setup Tool de Micro-Epsilon y arranca la grabacion al sensor ILD1302.
3. Presiona el boton RESET del Arduino. Vas a ver 5 segundos donde puedes entrar a modo MANUAL si envias una tecla. Si no haces nada, entra en modo AUTONOMO.
4. Cuenta 8 segundos: el Arduino esta quieto sincronizando. Esto es la pausa de referencia 0°.
5. Empieza a girar 30°, se queda 3 segundos quieto, gira 30° mas, etc. Hace 12 posiciones x 5 ciclos = 60 paradas en horario, luego repite en antihorario.
6. Cuando termine (el Arduino imprime 'Secuencia completada'), detiene la grabacion del Setup Tool y exporta a CSV.
7. Doble clic en lanzar.bat. Selecciona el CSV. Abre el Excel resultante.

3.2. Modo Semi-Automatico (boton fisico)

La version original. El operador pulsa un boton externo que envia un pulso al Arduino, y este avanza 90°. Cada parada debe durar al menos 3 segundos para que el script la detecte como medicion valida.

8. Inicia la grabacion del sensor.
9. Espera al menos 5 segundos quieto en 0° (referencia).
10. Pulsa el boton, el plato gira 90°.
11. Espera al menos 3 segundos (puedes esperar mas, no hay tope).
12. Repite hasta completar todos los ciclos previstos.
13. Detiene la grabacion, exporta CSV, doble clic en lanzar.bat.

3.3. Modo Manual (Monitor Serie)

Para pruebas puntuales, debug o cuando quieres mover el plato a una posicion arbitraria.

14. Abre el Arduino IDE y el Monitor Serie a 9600 baud.
15. Resetea el Arduino. En los primeros 5 segundos, envia cualquier tecla (por ejemplo 'H'). Quedas en modo MANUAL.
16. Usa los comandos abajo para mover.

Comandos en modo manual

H	Mueve PASO_ANGULO_DEG grados en sentido horario
A	Mueve PASO_ANGULO_DEG grados en sentido antihorario
+N	Mueve exactamente N pasos en sentido horario (ej: +500)
-N	Mueve exactamente N pasos en sentido antihorario
E	Habilita motor (energizado)

S	Deshabilita motor (libre, se puede girar a mano)
?	Muestra la configuración actual y el estado

4. Configuracion del Arduino (.ino)

Todas las variables ajustables estan en el bloque CONFIGURACION del archivo. No hay que tocar nada mas: las constantes derivadas (PASOS_VUELTA, POSICIONES) se calculan solas.

Variable	Default	Que hace / cuando cambiarla
PASOS_MOTOR_REV	200	Pasos por revolucion del motor. NEMA 1.8° = 200, NEMA 0.9° = 400. Cambia si reemplazas el motor.
RELACION_ENGRANAJES	40	El N de la relacion 1:N. Si pones engranajes 1:20, cambia a 20.
MICROSTEP	1	Configuracion del driver. Full=1, half=2, 1/4=4, 1/8=8, 1/16=16, 1/32=32. Sube si necesitas mas resolution.
PASO_ANGULO_DEG	30	Grados que avanza el plato en cada parada. Valores tipicos: 15, 30, 45, 60, 90, 120, 180.
CICLOS	5	Cuantas vueltas completas hace por sentido. Mas ciclos = mas datos para promediar, mas tiempo de ensayo.
SENTIDO_PRIMERO_ES_HORARIO	true	true=arranca horario, false=arranca antihorario.
MEDIR_AMBOS_SENTIDOS	true	true=mide horario Y antihorario (calcula reposicionamiento). false=solo el primero.
TIEMPO_ESPERA_INICIAL_MS	8000	Pausa inicial en ms. Sirve para que arranques la grabacion del sensor antes de que se mueva.
TIEMPO_ESPERA_POSICION_MS	3000	Pausa en cada parada (ms). Mas tiempo = mas datos por posicion = menor incertidumbre.
ESPERA_ENTRE_CICLOS_MS	0	Pausa adicional al terminar cada vuelta. Util si la maquina se calienta o necesitas inspeccion visual.
ESPERA_ENTRE_SENTIDOS_MS	3000	Pausa al cambiar de horario a antihorario. Da margen al sensor para detectar el cambio.
VEL_US	800	Microsegundos entre pulsos del motor. Menor = mas rapido pero puede perder pasos. Sube si vibra.
DETECT_MS	5000	Ventana al encender para escuchar una tecla y entrar en modo MANUAL.

REGLA DE ORO: Despues de modificar el .ino, recompila y sube el sketch al Arduino. Las constantes son #define en tiempo de compilacion: no se actualizan en caliente.

5. Configuracion del script Python (.py)

Las variables del bloque CONFIGURACION en el .py deben coincidir con las del .ino para que la asignacion de mediciones a posiciones sea correcta.

Variable	Default	Que hace
PASO_ANGULO	30	Grados por posicion. DEBE coincidir con PASO_ANGULO_DEG del .ino.
ANGULO_MAX	360	Recorrido total. 360 para vuelta completa, 180 para media vuelta.
REPETICIONES	5	Numero de ciclos. DEBE coincidir con CICLOS del .ino.
SENTIDO_PRIMERO	"Horario"	Texto: "Horario" o "Antihorario". Coincidir con el .ino.
MEDIR_AMBOS_SENTIDOS	True	Si False, solo procesa el primer sentido.
NUEVO_CERO_ENTRE_SENTIDOS	False	True si vuelves al 0° entre sentidos y tomas nueva referencia.
UMBRAL_STD	0.15	mm. Variacion maxima en la ventana para considerar 'quieto'. Sube si el sensor tiene ruido.
VENTANA_STD_SEG	0.20	s. Ventana del rolling std (cuanto tiempo evalua la 'quietud').
TIEMPO_QUIETO_MIN	2.50	s. Duracion minima de una pausa para contarla. Pon 2.5 si el Arduino pausa 3s.
TIEMPO_MARGEN	0.30	s. Recorta inicio y fin de cada pausa para evitar transitorios del movimiento.
TIEMPO_PAUSA_INICIO	7.00	s. Pausa MINIMA inicial para detectar la referencia 0°. Pon 7 si el Arduino espera 8s.
NOMBRE_SALIDA	"resultados_automatico.xlsx"	Nombre del Excel generado.

Coincidencia con el .ino: PASO_ANGULO, REPETICIONES, SENTIDO_PRIMERO y MEDIR_AMBOS_SENTIDOS deben ser identicos en .py y .ino. Si cambias el .ino, actualiza el .py.

6. Casos de uso: alternar para distintas maquinas

6.1. Tabla de combinaciones tipicas

La resolucion angular depende de tres factores: motor, engranajes y microstep. La formula es:

```
PASOS_VUELTA = PASOS_MOTOR_REV * RELACION_ENGRANAJES * MICROSTEP
Resolucion = 360 / PASOS_VUELTA (grados por paso)
```

Ejemplos para distintos hardware:

Configuracion	Motor (pasos/rev)	Engranajes (1:N)	Microstep	Pasos/vuelta	Resolucion (°/paso)
Actual (referencia)	200	40	1	8 000	0.045
Mas precision (1/2 step)	200	40	2	16 000	0.0225
Alta resolucion (1/16)	200	40	16	128 000	0.00281
Motor 0.9°	400	40	1	16 000	0.0225
Reductora 1:20	200	20	1	4 000	0.090
Reductora 1:100	200	100	1	20 000	0.018

6.2. Como elegir el paso angular (PASO_ANGULO_DEG)

- Pasos pequenos (15°, 30°): mas puntos de calibracion, ensayo mas largo. Util para mapas detallados de error angular.
- Pasos medios (45°, 60°): buen compromiso entre detalle y duracion.
- Pasos grandes (90°, 180°): ensayos rapidos, util para verificacion de simetria o pruebas funcionales.

Restriccion: 360 debe ser divisible entre PASO_ANGULO_DEG (asi tienes posiciones exactas a 0°, 30°, 60°... 360°). Valores validos: 1, 2, 3, 4, 5, 6, 8, 9, 10, 12, 15, 18, 20, 24, 30, 36, 40, 45, 60, 72, 90, 120, 180.

6.3. Como elegir el tiempo por posicion

El tiempo de quietud por posicion (TIEMPO_ESPERA_POSICION_MS en el .ino, TIEMPO_QUIETO_MIN en el .py) controla cuantas muestras se promedian para calcular la distancia en esa posicion.

- 3 segundos: minimo recomendado. Da unas 300 muestras a 100 Hz, suficiente para reducir ruido del sensor.
- 5-10 segundos: cuando el sensor es ruidoso o necesitas alta confianza en cada punto.
- Menor a 2 segundos: NO recomendado. El transitorio del movimiento (vibracion residual) puede contaminar la medicion.

Sincronizacion .ino / .py: Si subes TIEMPO_ESPERA_POSICION_MS en el .ino, baja TIEMPO_QUIETO_MIN en el .py para que detecte la pausa (siempre con margen, ej: 0.5s menos).

6.4. Como elegir velocidad (VEL_US)

- VEL_US = 800 (default): suave, sin riesgo de perder pasos. Da unos 3.2 s por 90°.
- VEL_US = 400: mas rapido, util si el ensayo es largo.
- VEL_US = 1500-2000: muy suave, util si el plato es pesado y hay vibraciones.

Regla: si oyes que el motor 'chilla' o pierde sincronia (no llega a la posicion esperada), sube VEL_US.

7. Logica interna del sistema

7.1. Compensacion acumulativa de pasos

El motor + engranajes da 8000 pasos por vuelta completa del plato (con la config default). Si queremos 12 posiciones (cada 30°), necesitamos $8000 / 12 = 666.66...$ pasos por posicion. Pero el motor solo entiende numeros enteros.

Solucion ingenua: redondear a 667 pasos siempre. Resultado: despues de 12 posiciones diste $12 * 667 = 8004$ pasos, no 8000. Error acumulado de 4 pasos = 0.18° por vuelta.

Solucion correcta (implementada): compensacion acumulativa. La funcion `pasosTramo(i)` reparte el sobrante de forma que la suma de los 12 tramos da exactamente 8000.

```
long pasosTramo(int i) {
    long a = (PASOS_VUELTA * (long)(i + 1)) / POSICIONES;
    long b = (PASOS_VUELTA * (long)i) / POSICIONES;
    return a - b;
}
```

Con 8000 pasos y 12 posiciones, esto produce la secuencia 666, 667, 667, 666, 667, 667, 666, 667, 667, 666, 667, 667 — que suma exactamente 8000. El plato siempre vuelve al 0° real despues de una vuelta completa.

7.2. Deteccion de periodos quietos en el Python

El script no sabe cuando el plato esta quieto: lo deduce de la distancia medida. Si la distancia varia poco durante un intervalo, asume que el plato no se esta moviendo.

Algoritmo:

17. Lee el CSV y obtiene una serie de (tiempo, distancia).
18. Calcula la desviacion estandar deslizante con una ventana de VENTANA_STD_SEG segundos.
19. Marca como 'quieto' todo punto donde la std < UMBRAL_STD (default 0.15 mm).
20. Agrupa los puntos quietos consecutivos en 'periodos'. Descarta los periodos con duracion < TIEMPO_QUIETO_MIN.
21. Recorta TIEMPO_MARGEN segundos al inicio y al fin de cada periodo (para evitar transitorios).
22. Calcula promedio, desviacion estandar y numero de muestras de cada periodo.

7.3. Asignacion de periodos a posiciones

Una vez detectados los periodos quietos, el script los asigna en orden a la secuencia configurada:

23. El PRIMER periodo de duracion $\geq$ TIEMPO_PAUSA_INICIO se toma como referencia 0°. Su promedio es la distancia de referencia.
24. Los siguientes periodos se asignan en orden: posicion 1 (30°), posicion 2 (60°), ... posicion 12 (360°), ciclo 2 posicion 1, etc.

25. Si MEDIR_AMBOS_SENTIDOS=true, despues de terminar el sentido horario continua con el antihorario.
26. Si NUEVO_CERO_ENTRE_SENTIDOS=true, espera otra pausa larga entre sentidos como nueva referencia.
27. Para cada posicion calcula: $\text{error} = \text{distancia_medida} - \text{distancia_referencia}$.

7.4. Calculo de resultados

Con todos los puntos asignados, el script genera dos tablas principales:

Tabla de calibracion

Por cada angulo y sentido, promedia los errores de todos los ciclos y calcula la desviacion estandar entre ciclos. Si la desviacion es alta = baja repetibilidad en ese angulo.

Tabla de reposicionamiento (solo si se miden ambos sentidos)

Por cada angulo, calcula la diferencia entre la distancia medida girando horario vs antihorario. Esa diferencia es el error de reposicionamiento: cuanto cambia la posicion segun la direccion de aproximacion. Es un indicador clasico de holgura mecanica.

8. Solucion de problemas

El motor no se mueve

- Verifica que ENABLE (pin 8) esta a LOW (motor energizado). Envia 'E' por el monitor serie.
- Mide los pulsos en Y_STEP (pin 3) con osciloscopio o LED.
- Revisa la alimentacion del driver (12-24 V tipico).

El motor vibra pero no avanza

- Posible falla de cableado: revisa los pares de bobinas (A+, A-, B+, B-).
- Sube VEL_US (ej: 1500) para reducir el chance de perder pasos.
- Comprueba la corriente del driver (Vref del A4988/DRV8825).

El plato no llega exactamente a 360° despues de una vuelta

- Verifica que PASOS_MOTOR_REV, RELACION_ENGRANAJES y MICROSTEP coinciden con tu hardware real.
- Si el error es por backlash mecanico, no hay solucion electronica: revisa engranajes.

Python no detecta todas las pausas

- Sube UMBRAL_STD (ej: 0.20 o 0.25 mm) si el sensor tiene ruido.
- Baja TIEMPO_QUIETO_MIN si las pausas son cortas.
- Revisa que el operador no este moviendo demasiado lento (el script puede no distinguir el movimiento de la quietud).

Python detecta MAS pausas de las esperadas

- Baja UMBRAL_STD (ej: 0.10 mm) para ser mas estricto.
- Sube VENTANA_STD_SEG (ej: 0.30 s) para suavizar la deteccion.

- Verifica que el operador no este haciendo pausas intermedias accidentales.

El Excel sale con N/D en muchas celdas

- Significa que faltan periodos detectados. Compara 'Slots con datos: X / Y' en el log del script.
- Probablemente el ensayo se interrumpio antes de completar todos los ciclos.
- Revisa el CSV en el Setup Tool: confirma que las posiciones realmente se midieron.

9. Resumen rapido

Cinco cosas para recordar:

28. Edita SOLO el bloque CONFIGURACION en ambos archivos (.ino y .py). El resto se calcula solo.
29. PASO_ANGULO, REPETICIONES, SENTIDO_PRIMERO y MEDIR_AMBOS_SENTIDOS deben coincidir entre .ino y .py.
30. Despues de modificar el .ino, RECOMPILA Y SUBE el sketch. No se actualiza en caliente.
31. Para ensayos rigurosos usa modo AUTONOMO. El .py funciona igual para los tres modos.
32. Si cambias hardware, ajusta PASOS_MOTOR_REV, RELACION_ENGRANAJES y MICROSTEP. Verifica con una vuelta completa que el plato vuelve al 0°.

10. Anexo

- Descarga de todos los programas y documentación:
<https://github.com/ApocalypZmbe/DSMP>